SVKM'S NMIMS Deemed to be UNIVERSITY | CHANDIGARH

SVKM'S NMIMS

Mukesh Patel School of Technology Management & Engineering

Chandigarh Campus

PROJECT REPORT

MESSMATE

Eat Better. Choose Smarter.

Submitted by

Harkirat Kaur (T021) SAP ID: 70022400577

Aasandi (T001) SAP ID: 70022400133

in partial fulfilment of

BACHELOR OF TECHNOLOGY

IN

COMPUTER ENGINEERING

Subject: Mobile Application Development (MAD) | Faculty: Dr. Preetjot Kaur

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF TECHNOLOGY MANAGEMENT AND ENGINEERING, CHANDIGARH, 160114, U.T. (INDIA)

ABSTRACT

MessMate is an Android application designed to make mess-meal discovery and selection more organized for students. In a typical hostel or campus mess environment, meal information may be difficult to compare, students may have different dietary preferences, and feedback may not be stored in a structured way. MessMate brings meal browsing, preference capture, meal details, ratings, comments, and recommendation-related logic into a single mobile application.

The application is developed using Java for application logic and XML for layouts. Firebase Authentication supports account creation and sign-in, while Cloud Firestore stores user profiles, meal information, ratings, preferences, and date-specific menu selections. The app uses Activities for full-screen workflows, Fragments for the main navigation sections, model classes for structured data, adapters for list presentation, and repository/view-model classes to support separation of responsibilities.

A rule-based RecommendationEngine is included in the source code. It assigns scores based on food-type match, spice preference, favourite-food text matches, and stored meal ratings, while excluding unavailable meals and meals matching disliked terms. The current RecommendationFragment, however, sorts available meals by stored rating rather than invoking this engine; this distinction is documented to accurately represent the current implementation. The project demonstrates how Android UI components and a cloud-backed data layer can be combined to support a practical student-facing application.

Keywords: Android, Java, XML, Firebase Authentication, Cloud Firestore, meal management, user preferences, ratings, rule-based recommendation.

SVKM'S NMIMS Deemed to be UNIVERSITY | CHANDIGARH

TABLE OF CONTENTS

1. Introduction

2. Problem Definition and Objectives

3. Existing System and Proposed Solution

4. Requirements Analysis

5. System Design and Architecture

6. Technology Stack

7. Application Modules and Features

8. Database Design

9. Recommendation Engine

10. Implementation Details

11. Testing and Validation Plan

12. Results and Discussion

13. Limitations and Future Enhancements

14. Conclusion

References

Appendix A: Important Classes

Appendix B: Suggested Screenshots for Final Submission

1. INTRODUCTION

1.1 Background

Mobile applications are widely used to make everyday information easier to access. For students living in hostels or using a campus mess, meal-related decisions can involve checking the available menu, understanding meal categories, considering food type and spice preferences, and deciding whether to provide feedback. When these details are spread across notices, verbal announcements, or informal messages, the process becomes less convenient.

1.2 Project Overview

MessMate is a student-oriented Android application that centralizes meal discovery and related interactions. The app provides authentication, a home area, a menu, a recommendation-oriented screen, a profile section, meal details, preference management, and meal-rating functionality. It also contains administration-oriented screens for menu management and rating analytics.

1.3 Purpose

The purpose of MessMate is to demonstrate the design and development of a practical Android application that combines native Android UI components with a cloud-hosted backend. The project also provides an opportunity to apply object-oriented programming, event-driven programming, data modelling, asynchronous database operations, and modular application design.

1.4 Scope

- Provide account creation and email/password sign-in using Firebase Authentication.
- Capture profile details and food preferences.
- Display meal information and allow navigation to meal details.
- Collect ratings and written feedback for meals.
- Store and retrieve application data through Cloud Firestore.
- Provide an administrator-oriented workflow for managing meals and date-specific menu selections.
- Include rule-based recommendation logic and a recommendation-oriented interface.

1.5 Intended Users

User Type	Main Need	Relevant Features
Student	Discover meals and provide feedback	Home, menu, meal details, preferences, ratings, profile
Administrator / mess manager	Maintain meal and menu information	Admin login/dashboard, meal management, menu management, analytics

2. PROBLEM DEFINITION AND OBJECTIVES

2.1 Problem Statement

Students may have limited visibility into the available mess meals and may find it difficult to compare meal options against their preferences. Dietary type, spice tolerance, favourite foods, dislikes, and previous ratings can influence meal choices. In addition, unstructured feedback is difficult to review consistently. A mobile application can bring these interactions together and provide a more organized way to browse meals and record feedback.

2.2 Project Objectives

- Develop a native Android application using Java and XML.
- Provide a clear entry flow that directs users to sign-in or the main application based on authentication state.
- Store account and profile information in Firebase services.
- Display meals using list-based UI components and provide a detailed view for each selected meal.
- Allow users to record ratings and comments associated with a meal and user account.
- Capture food preferences, including food type, spice level, favourite foods, dislikes, and cuisine selections.
- Implement a separate, explainable rule-based scoring engine for meal recommendations.
- Provide basic administration screens for meal/menu maintenance and rating summaries.
- Organize the source code into packages to improve readability, reuse, and maintainability.

2.3 Expected Outcomes

The expected outcome is a working Android application prototype that demonstrates a complete user journey from authentication to meal discovery and feedback, together with cloud data storage

and a foundation for personalized meal selection. The quality of the final outcome should be verified through functional testing on the final build.

2.4 Project Boundaries

MessMate is not a clinical nutrition application and does not provide medically validated dietary advice. The recommendation engine is rule-based rather than trained through machine learning. The project does not, by itself, establish that ratings automatically update meal averages or that the date-specific menu is the source used by the student menu screen; these are identified as integration opportunities.

3. EXISTING SYSTEM AND PROPOSED SOLUTION

3.1 Existing Approach

In a basic mess environment, menus may be communicated using notice boards, printed lists, or informal messages. Feedback may be verbal or recorded without a consistent structure. These methods can work for small groups, but they may not provide an easily searchable history or a convenient way to connect feedback with a specific meal.

3.2 Challenges

- Meal details may be scattered or not consistently presented.
- Students have different preferences that are not reflected in a common menu list.
- Feedback can be difficult to organize by meal, user, rating, and date.
- Administrators may lack a single view for maintaining meal data and reviewing rating distribution.

3.3 Proposed Solution

MessMate provides a single Android interface backed by Firebase services. Students can authenticate, view meal lists, open meal details, set preferences, and submit feedback. Administrative screens support meal maintenance and date-specific menu selection. A scoring engine is included to illustrate how available meal data and saved preferences can be converted into a ranked list.

3.4 Benefits

- Centralized access to meal information.
- Structured storage of user profiles, preferences, meals, and feedback.
- Consistent association of a rating with the meal ID and user ID.
- Modular code organization that supports maintenance and future extension.
- Transparent recommendation rules that can be explained and adjusted.

4. REQUIREMENTS ANALYSIS

4.1 Functional Requirements

ID	Requirement	Description
FR-01	Authentication	Users can sign up, log in, log out, and request a password reset.
FR-02	Profile storage	Profile information is saved against the authenticated user UID.
FR-03	Preference capture	Users can save food type, spice level, favourite/disliked foods, and cuisine selections.
FR-04	Meal browsing	The app retrieves and displays meal data, including category, description, price, rating, and availability where present.
FR-05	Meal details	A selected meal can be opened using its Firestore document ID.
FR-06	Feedback	Users can submit an overall rating, additional rating dimensions, a comment, and a timestamp.
FR-07	Recommendation support	The project includes rule-based scoring logic based on meal and preference attributes.
FR-08	Administration	Admin-oriented screens support dashboard access, meal management, menu selection, and rating summaries.
FR-09	Cloud persistence	Application records are read from and written to Cloud Firestore.
FR-10	Error feedback	Asynchronous success/failure callbacks provide the basis for displaying outcomes to users.

4.2 Non-Functional Requirements

Requirement	Expectation
Usability	Screens should use clear labels, predictable navigation, and validation messages.
Performance	Database operations should be asynchronous and should not block the main UI thread.
Reliability	Failed requests should be handled with clear error states and retry-friendly workflows.
Maintainability	Activities, Fragments, models, adapters, repositories, and utilities should have distinct responsibilities.
Security	Firestore access must be protected by Firebase Security Rules and authenticated access where appropriate.
Compatibility	The app should be tested on supported Android versions and screen sizes configured by the project.
Data integrity	Ratings and menu entries should retain stable references to their associated meal documents.

4.3 Hardware and Software Requirements

Category	Requirement
Development environment	Android Studio with a compatible Android SDK and Gradle setup
Language and UI	Java and XML layout resources
Backend services	Firebase Authentication and Cloud Firestore
Testing device	Android emulator or compatible Android phone
Connectivity	Internet connection for Firebase-backed operations
Version control / packaging	Project source folder and Gradle build files; repository usage may be added if applicable

4.4 Assumptions and Constraints

The application assumes that Firebase is configured correctly for the Android package and that the device has network access for cloud operations. The app's actual behaviour depends on the deployed Firestore schema, security rules, and Firebase project configuration. Sample data initialization is intended for development/demo use and should not overwrite production data.

5. SYSTEM DESIGN AND ARCHITECTURE

5.1 Architectural Overview

MessMate follows a layered, component-oriented structure. XML resources define layouts; Activities and Fragments manage screen interactions; model classes represent structured data; adapters bind lists to RecyclerView rows; repositories and helper classes provide reusable data-access functionality; Firebase Authentication manages identity; and Cloud Firestore stores cloud records. ViewModel classes are present in the project, although the current source does not route every screen through a ViewModel or repository.

Layer / Component	Responsibility	Examples
Presentation	Screens, user input, navigation	Activities, Fragments, XML layouts
List presentation	Display and bind collections of data	MealAdapter, RecommendationAdapter, FeedbackAdapter
Domain logic	Preference matching and scoring	RecommendationEngine
Data models	Represent users, meals, preferences, ratings	Meal, Preference, Rating, Recommendation
Data access	Centralize reusable reads and writes	MealRepository, RatingRepository, UserRepository
State support	Hold/expose screen data where integrated	MealViewModel, ProfileViewModel, UserViewModel
Cloud services	Authentication and persistent records	Firebase Authentication, Cloud Firestore
Utilities	Shared constants and initialization	Constants, FirebaseHelper, DataSeeder

5.2 High-Level Data Flow

Step	Component	Action
1	User interface	User taps a control or enters information.
2	Activity / Fragment	Validates input and starts an operation or navigation.
3	Repository / Firebase SDK	Builds a Firestore query or write request; some screens also call Firestore directly.
4	Cloud Firestore	Applies configured security rules and processes the read/write.
5	Asynchronous callback	Returns a success result or failure information.
6	Model / adapter / UI	Converts data, refreshes the list, displays details, or reports an error.

5.3 Student Application Flow

SplashActivity → LoginActivity or MainActivity → Home / Menu / For You / Profile → Meal Detail → Rate Meal

A new user can proceed through SignupActivity and PreferenceActivity before entering the main application. SplashActivity checks the current Firebase Authentication user after a short delay. If a signed-in user exists, the app opens MainActivity; otherwise, it opens LoginActivity.

5.4 Administrator Flow

The source includes administrator-oriented screens such as AdminLoginActivity, AdminDashboardActivity, AdminHomeFragment, ManageMenuFragment, and AnalyticsFragment. These support an administrative path for dashboard access, menu/meal management, and rating summaries. The exact authorization guarantees depend on the implementation and Firebase Security Rules; hiding an admin screen in the UI alone is not a security boundary.

5.5 Navigation and Data Passing

MainActivity uses bottom navigation to replace the content of mainFragmentContainer with the selected Fragment. When a meal is selected, an Intent can pass values such as mealId and mealName to MealDetailActivity. The stable mealId is then used to retrieve the matching Firestore document and to associate a submitted rating with the correct meal.

6. TECHNOLOGY STACK

Technology	Role in MessMate
Java	Implements Activity/Fragment behaviour, validation, navigation, data handling, and recommendation rules.
XML	Defines screen layouts, row layouts, menus, themes, and drawable resources.
Android Activities	Manage full-screen flows such as login, sign-up, preferences, meal detail, and rating.
Android Fragments	Provide reusable sections for home, menu, recommendations, profile, and administration.
RecyclerView and adapters	Display scrollable collections by binding model data to item layouts.
Firebase Authentication	Supports email/password account creation, sign-in, sign-out, and password-reset requests.

Cloud Firestore	Stores user, meal, preference, rating, and daily-menu records.
LiveData / ViewModel	Classes are present to support observable screen state where used.
Gradle	Manages Android build configuration and dependencies.

6.1 Why These Technologies Were Selected

Java and XML align with traditional Android application development and make the relationship between UI layouts and event-handling code explicit. Firebase provides managed authentication and a cloud document database, reducing the need to build a custom authentication server or database API for the prototype. RecyclerView and adapters support efficient list rendering, while model and repository classes make data easier to represent and reuse.

7. APPLICATION MODULES AND FEATURES

7.1 Splash and Authentication Module

SplashActivity acts as the application entry point. It triggers sample-data initialization and then checks Firebase Authentication after approximately 1.8 seconds. LoginActivity accepts email and password, checks for empty fields, and calls signInWithEmailAndPassword. A successful sign-in opens MainActivity; a failed attempt displays a message. The login flow also includes navigation to sign-up and password-reset functionality.

7.2 Registration Module

SignupActivity creates a Firebase Authentication account. Once the authenticated user UID is available, profile information is written to users/{uid} in Firestore. Authentication and profile persistence are separate operations: an account can be created even if the subsequent Firestore write fails, so robust error handling should communicate that distinction.

7.3 Preference Module

PreferenceActivity captures the user's food type, spice level, favourite foods, disliked foods, and cuisine selections. Preference data is stored in the user's Firestore document using a merge-style write so existing profile fields can be preserved. These values provide inputs for the rule-based recommendation engine.

7.4 Home Module

HomeFragment presents a greeting, a special meal area, and shortcuts. It acts as a convenient starting point for the student workflow. The information shown depends on the records available in Firestore and the fields used by the fragment.

7.5 Menu Module

MenuFragment loads available meal records and presents them by category. Meal cards are displayed through list adapters. Selecting a meal passes its identifier to MealDetailActivity, which retrieves the full record. In the current implementation described by the project walkthrough, MenuFragment queries the meals collection directly; it does not yet consume the date-specific dailyMenus selection written by ManageMenuFragment.

7.6 Meal Detail Module

MealDetailActivity displays information about the selected meal, such as its name, category, description, and other available attributes. The meal ID is passed through an Intent, allowing the detail screen to retrieve the corresponding Firestore document and the rating screen to reference the same meal.

7.7 Rating and Feedback Module

RateMealActivity collects an overall star rating, additional taste/quality/quantity values through SeekBars, a comment, and the current timestamp. A rating record includes mealId, userId, rating, comment, and createdAt, with additional rating dimensions where supplied. The record is added to the ratings collection using an automatically generated document ID. Submitting a rating currently creates a feedback record; it does not automatically recalculate and update the rating field on the meal document.

7.8 Recommendation Module

RecommendationFragment displays a recommendation-oriented list. The current fragment sorts available meals by their stored rating. Separately, RecommendationEngine.java implements preference-aware scoring and explanation logic. As these paths are not yet connected, the report distinguishes the existing screen behaviour from the standalone scoring engine.

7.9 Profile Module

ProfileFragment displays profile information and provides navigation for preference management and logout. Logout should clear the Firebase Authentication session and return the user to an unauthenticated entry point.

7.10 Administration Module

The project includes administrator-oriented screens for dashboard information, meal management, date-specific menu selection, and rating analytics. ManageMenuFragment writes a dailyMenus/{date} document containing category-to-meal selections and an update timestamp. AnalyticsFragment summarizes rating distribution. Data integrity and access control must be enforced through backend rules, not only through screen navigation.

7.11 Data Initialization Module

DataSeeder checks whether the meals collection is empty and, when appropriate, inserts sample meals and ratings. This supports a demonstrable first-run experience. It should be restricted to development/demo environments or made safely idempotent so it cannot overwrite real operational data.

8. DATABASE DESIGN

8.1 Cloud Firestore Overview

Cloud Firestore is a NoSQL document database. In MessMate, each collection groups related documents, and each document stores fields as key-value pairs. The application uses document IDs to identify records and connect related information.

8.2 Main Collections

Collection / Document	Representative Fields	Purpose
users/{uid}	name, email, hostel, room, foodType, role, preferences	Stores profile and preference information for an authenticated account.
meals/{mealId}	name, category, description, price, rating, available, isSpecial, isRecommended	Stores meal details and attributes used in list display and scoring.
ratings/{autoId}	mealId, mealName, userId, rating, taste, quality, quantity, comment, createdAt	Stores feedback associated with a meal and a user.
dailyMenus/{date}	meals map, updatedAt	Stores selected meal IDs by category for a specific date.

8.3 Data Relationships

A user document is identified by the Firebase Authentication UID. A rating stores userId and mealId values to associate the feedback with its author and target meal. The mealId should match the ID of a document in the meals collection. A daily menu stores selected meal IDs grouped by category; the application can resolve those IDs to meal documents when the menu integration is implemented.

8.4 Example Meal Record

Field	Example Value	Data Type
name	Aloo Paratha & Curd	String
category	Breakfast	String
price	45.0	Number
rating	4.7	Number
available	true	Boolean
isRecommended	true	Boolean

The example above illustrates the field structure used in the project walkthrough. Actual stored values depend on the current Firestore dataset. Field names and data types must remain consistent between write operations, queries, and model conversion.

8.5 Read and Write Operations

The Firebase SDK performs Firestore operations asynchronously. A typical read requests a collection or query, receives a QuerySnapshot in a success callback, converts each DocumentSnapshot to a model object, and passes the results to a UI component. Failure callbacks should handle network, permission, or data-conversion errors. When converting a document to a model, assigning doc.getId() to the model's ID field is important because the document ID may not be stored as a regular field.

8.6 Security and Privacy Considerations

- Require authentication for operations that should only be available to signed-in users.
- Use Firestore Security Rules to restrict profile reads/writes and validate user ownership of ratings.
- Restrict meal and menu write access to authorized administrator accounts.

- Do not rely on client-side role checks alone to secure administrative data.
- Collect only necessary profile data and avoid exposing one user's private information to another.
- Keep Firebase configuration and any privileged credentials out of public source repositories.

9. RECOMMENDATION ENGINE

9.1 Overview

RecommendationEngine.java implements an explainable, rule-based scoring method. It is not a trained machine-learning model. Its purpose is to rank eligible meals using a combination of a base score, preference matches, and the stored meal rating.

9.2 Inputs

- List of meal objects.
- Preferred food type.
- Preferred spice level.
- Favourite-food text.
- Disliked-food text.

9.3 Eligibility Filtering

The engine skips null or unavailable meals and excludes meals whose name or description contains a disliked term. This prevents clearly unsuitable candidates from receiving a high score. The quality of text matching depends on how consistently meal names, descriptions, and preference terms are entered.

9.4 Scoring Formula

The implementation starts each eligible meal at 50 points, then adds preference and rating contributions:

Score = min(100, 50 + F + S + L + R)

Component	Rule	Maximum Contribution
Base score	Every eligible meal starts at 50.	50 base points
Food-type match (F)	Add 20 if the meal food type matches the preferred food type.	20

Spice match (S)	Add 10 if spice level matches the preferred level.	10
Favourite match (L)	Add 10 if meal name/description matches favourite-food text.	10
Rating contribution (R)	Add meal rating × 2, bounded between 0 and 10.	10
Cap	Final score is capped at 100.	100 total maximum

9.5 Ranking and Explanation

Eligible meals are sorted in descending order of score. The Recommendation model contains a score and an explanation field so the user can be shown why a meal was ranked highly. Explanations can mention matching food type, spice level, favourite terms, or rating contribution.

9.6 Example Calculation

Condition	Contribution
Base score	50
Preferred food type matches	+20
Preferred spice matches	+10
Favourite text matches	+10
Meal rating is 4.2; rating contribution = 4.2 × 2	+8.4
Raw score	98.4
Final score after cap	98.4 / 100

9.7 Current Integration Status

The standalone RecommendationEngine is present in the source. The current RecommendationFragment, as documented in the project presentation, separately sorts available meals by stored rating and does not yet call the engine. To make preference-aware recommendations visible in the UI, the fragment or its ViewModel should load saved preferences and available meals, invoke RecommendationEngine, and render the returned Recommendation objects.

10. IMPLEMENTATION DETAILS

10.1 Project Package Structure

Package	Purpose
activities/	Full-screen flows: splash, login, signup, main navigation, preferences, meal details, ratings, and admin screens.
fragments/	Student and administrator screen sections, including home, menu, recommendations, profile, menu management, and analytics.
models/	Data objects for Meal, Preference, Rating, MealRating, Recommendation, User, and UserProfile.
adapters/	RecyclerView adapters that bind meals, recommendations, and feedback to row layouts.
repository/	Reusable data-access classes for meals, ratings, and users.
viewmodel/	ViewModel classes for meal, profile, recommendation, and user state where integrated.
recommendation/	Rule-based meal scoring and ranking logic.
utils/	Shared constants, Firebase helper logic, and sample data initialization.

10.2 Activity Lifecycle and View Binding

An Activity typically performs its initial setup in onCreate(). It calls setContentView() to inflate the relevant XML layout, uses findViewById() to connect Java references to view IDs, and registers click listeners to respond to user actions. Intents and startActivity() support navigation between Activities, while finish() removes an Activity from the back stack when returning to it would be undesirable.

10.3 RecyclerView and Adapters

RecyclerView displays scrollable collections efficiently by reusing row views. An adapter creates or reuses a ViewHolder and binds the data object at a given position to the row's TextViews and other controls. MealAdapter is associated with meal rows, RecommendationAdapter with recommendation rows, and FeedbackAdapter with feedback rows. This separation keeps list presentation code reusable.

10.4 Asynchronous Firebase Operations

Firestore operations should not block the Android main thread. The application uses success and failure listeners to respond when requests complete. The UI should display loading, empty, success, and failure states where appropriate. Callbacks should also avoid updating a screen that is no longer active.

10.5 Navigation and Identifiers

Stable identifiers such as mealId are passed between screens rather than relying only on display names. Display names can change or be duplicated; a document ID provides a consistent reference for loading details and associating feedback. Firestore field keys are determined by the strings used in map writes or by model serialization.

10.6 Error Handling

- Validate required fields before making network requests.
- Disable submit buttons while a request is in progress to reduce duplicate operations.
- Show meaningful messages when authentication or database operations fail.
- Handle empty query results separately from request failures.
- Check nullable model conversions and missing Firestore fields.
- Use secure Firestore rules and validate data on the backend boundary.

11. TESTING AND VALIDATION PLAN

The following test cases are proposed for validating the final application. They are a test plan, not a claim that every case has already been executed. Record the actual result and evidence after testing the final APK or emulator build.

Test ID	Scenario	Expected Result	Actual Result
TC-01	Launch app with no authenticated user	Splash routes to LoginActivity.	To be filled after testing
TC-02	Launch app with authenticated user	Splash routes to MainActivity.	To be filled after testing
TC-03	Submit login with empty fields	Validation prevents the request and indicates missing input.	To be filled after testing
TC-04	Login with valid credentials	Authentication succeeds and main screen opens.	To be filled after testing
TC-05	Login with invalid credentials	Failure message is displayed; app remains	To be filled after testing

		usable.	
TC-06	Create a new account	Authentication account is created and profile write is attempted.	To be filled after testing
TC-07	Save preferences	Preference fields are persisted to the correct user document.	To be filled after testing
TC-08	Load menu	Available meals are retrieved and displayed.	To be filled after testing
TC-09	Open a meal card	Correct meal details load using the meal document ID.	To be filled after testing
TC-10	Submit rating and comment	A rating record with user, meal, score, and timestamp is created.	To be filled after testing
TC-11	Firestore request denied or offline	Failure is handled without crashing the screen.	To be filled after testing
TC-12	Run recommendation engine	Unavailable/disliked meals are excluded and eligible meals are ranked.	To be filled after testing
TC-13	Save date-specific menu as admin	A dailyMenus document is written for the selected date.	To be filled after testing
TC-14	Unauthorized attempt to edit admin data	Firestore rules reject the write.	To be filled after testing

11.1 Testing Types

- Functional testing: verify each user-visible feature against its requirement.
- UI testing: check layout, scrolling, validation, navigation, and small-screen behaviour.
- Integration testing: verify authentication, Firestore reads/writes, and model conversion.
- Negative testing: test invalid credentials, empty inputs, unavailable network, missing fields, and permission-denied responses.
- Security testing: confirm Firestore rules prevent unauthorized profile, rating, meal, and menu access.
- Regression testing: repeat key user flows after changes to navigation, schema, or recommendation logic.

12. RESULTS AND DISCUSSION

12.1 Implemented Project Structure

The submitted source includes the main Android components required for a multi-screen application: Activities, Fragments, XML layouts, models, adapters, repositories, ViewModels, utility classes, and a separate recommendation package. This organization demonstrates modular design and makes the responsibilities of major components easier to explain.

12.2 Cloud-Backed Workflows

Firebase Authentication provides the identity workflow, while Firestore provides document storage for profiles, meals, ratings, and daily menu data. The project uses asynchronous callbacks to process database results. A user UID and meal document ID are used as stable references when storing related information.

12.3 Recommendation Capability

The rule-based scoring engine demonstrates a transparent approach to personalized ranking. Its score contributions can be inspected and adjusted without training a model. However, the UI integration remains incomplete in the current source description: RecommendationFragment sorts by stored rating instead of invoking the preference-aware engine.

12.4 Honest Evaluation of Current State

Area	Current state / note
Authentication	Firebase Authentication calls are included in the app workflow. Verify live configuration and error states during final testing.
Menu display	MenuFragment queries meals directly. Date-specific daily menu selection is not yet connected to this screen.
Ratings	Rating submission creates a ratings document. Meal aggregate rating is not automatically updated by this flow.
Latest feedback	Newest-first display requires an explicit timestamp sort in the relevant query.
Recommendations	Scoring engine exists but is not invoked by RecommendationFragment in the current walkthrough.
Security	Firebase Security Rules must enforce access restrictions; client-side UI checks are insufficient.
Sample data	DataSeeder is intended for development/demo setup and should be guarded against production use.

12.5 Overall Discussion

MessMate demonstrates the key concepts of mobile application development: UI construction, navigation, input validation, cloud authentication, document-based persistence, list rendering, modular code organization, and rule-based decision logic. The main improvement opportunities concern end-to-end integration and validation rather than the addition of more screens alone.

13. LIMITATIONS AND FUTURE ENHANCEMENTS

13.1 Current Limitations

- The recommendation engine is rule-based and uses simple text matching; it does not learn from user behaviour.
- The recommendation UI and the standalone scoring engine are not yet connected.
- The menu screen does not yet use the date-specific dailyMenus document written by the administration workflow.
- Rating submission does not automatically update an aggregate rating field on the meal document.
- Newest-first feedback display needs an explicit createdAt ordering.
- The quality of the displayed data depends on Firestore schema consistency and sample-data initialization.
- Security depends on correct Firebase Security Rules and role validation.
- The final application should be tested on the target Android version and device sizes before claiming production readiness.

13.2 Future Enhancements

- Integrate RecommendationEngine with RecommendationFragment or RecommendationViewModel and display the score explanation.
- Connect MenuFragment to dailyMenus/{date}, resolving selected meal IDs into meal details for the chosen date.
- Calculate aggregate ratings from individual feedback using a trusted backend function or a carefully secured transaction strategy.
- Add explicit timestamp ordering and pagination to feedback history.

- Add loading indicators, empty states, retry actions, and clearer error messages to all cloud-backed screens.
- Strengthen Firestore Security Rules for profile ownership, student rating creation, and administrator-only writes.
- Add validation for rating ranges, required fields, text length, and duplicate submissions.
- Provide filters for meal category, food type, spice level, availability, and dietary requirements.
- Improve accessibility with content descriptions, adequate contrast, scalable text, and touch targets.
- Add automated tests for scoring rules, model parsing, and key navigation flows.
- Add an administrator audit trail for meal/menu changes and a safe production data migration process.
- Consider nutrition/allergen information only when accurate data sources and appropriate verification are available.

13.3 Prioritized Improvements

Priority	Improvement	Reason
High	Connect the recommendation engine to the UI	Makes saved preferences affect visible ranking.
High	Enforce Firestore Security Rules	Protects user data and administrative operations.
High	Connect dailyMenus to the student menu	Aligns the displayed menu with administrator selections.
Medium	Maintain aggregate ratings and sort feedback	Improves consistency of rating displays.
Medium	Add loading/error/empty states	Improves usability during network and database failures.
Low / future	Advanced personalization and analytics	Can be considered after the core workflows are reliable.

14. CONCLUSION

MessMate is an Android-based meal discovery and feedback application developed using Java, XML layouts, Firebase Authentication, and Cloud Firestore. It brings together authentication, profile and preference capture, meal browsing, meal details, ratings, and administration-oriented functionality. Its modular structure separates screens, models, adapters, data access, utilities, and recommendation logic.

The project also illustrates the importance of documenting implementation status precisely. The source includes a rule-based recommendation engine, but the current recommendation fragment sorts by stored rating; the date-specific menu is written by the administration workflow but is not yet the direct source for the student menu; and rating submissions do not automatically update the stored meal rating. These observations provide a concrete roadmap for integration and testing.

Overall, MessMate provides a useful foundation for applying Mobile Application Development concepts to a practical campus problem. With the recommended integration, security, and testing improvements, it can become a more consistent and reliable meal-management prototype.

REFERENCES

I. Android Developers. “Activities overview.” https://developer.android.com/guide/components/activities

II. Android Developers. “Fragments.” https://developer.android.com/guide/fragments

III. Android Developers. “RecyclerView.” https://developer.android.com/develop/ui/views/layout/recyclerview

IV. Android Developers. “Intents and intent filters.” https://developer.android.com/guide/components/intents-filters

V. Firebase Documentation. “Authenticate with Firebase using Password-Based Accounts on Android.” https://firebase.google.com/docs/auth/android/password-auth

VI. Firebase Documentation. “Get started with Cloud Firestore on Android.” https://firebase.google.com/docs/firestore/quickstart

VII. Firebase Documentation. “Understand Cloud Firestore Security Rules.” https://firebase.google.com/docs/firestore/security/get-started

APPENDIX A: IMPORTANT CLASSES

Class / File	Responsibility
SplashActivity.java	Entry point; initializes demo data and routes based on authentication state.
LoginActivity.java	Email/password login and password-reset workflow.
SignupActivity.java	Creates an account and writes profile data.
MainActivity.java	Hosts bottom navigation and replaces the main Fragment.
PreferenceActivity.java	Collects and saves food preferences.
MealDetailActivity.java	Loads and displays a selected meal.
RateMealActivity.java	Collects rating dimensions and comments and writes a rating record.
HomeFragment.java	Home greeting, special meal, and shortcuts.
MenuFragment.java	Loads and displays available meals.
RecommendationFragment.java	Displays a rating-sorted recommendation-oriented list in the current implementation.
ProfileFragment.java	Displays profile information and supports profile-related navigation/logout.
ManageMenuFragment.java	Writes date-specific menu selections.
AnalyticsFragment.java	Summarizes rating distribution.
Meal.java / Preference.java / Rating.java	Structured representations of core data.
Recommendation.java / RecommendationEngine.java	Represent a recommendation and calculate its rule-based score.
MealRepository.java / RatingRepository.java / UserRepository.java	Reusable data-access classes.
DataSeeder.java	Inserts sample meals and ratings when the meals collection is empty.

APPENDIX B: SCREENSHOTS FOR FINAL SUBMISSION

The following screenshots were supplied for the MessMate application. They illustrate the visible student-facing screens and are included as figures in the project report.

Figure 1. MessMate splash screen

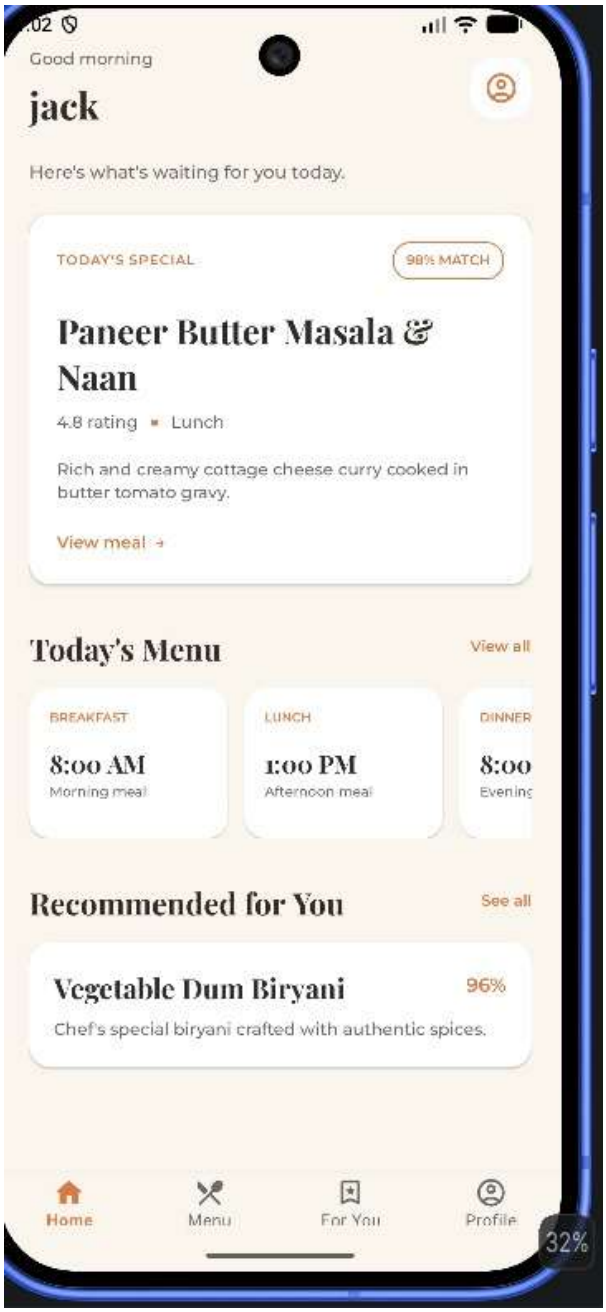
The splash screen presents the MessMate name, food-themed logo, and tagline "Eat Better. Choose Smarter."



Source: screenshot supplied by the project team.

Figure 2. Home dashboard

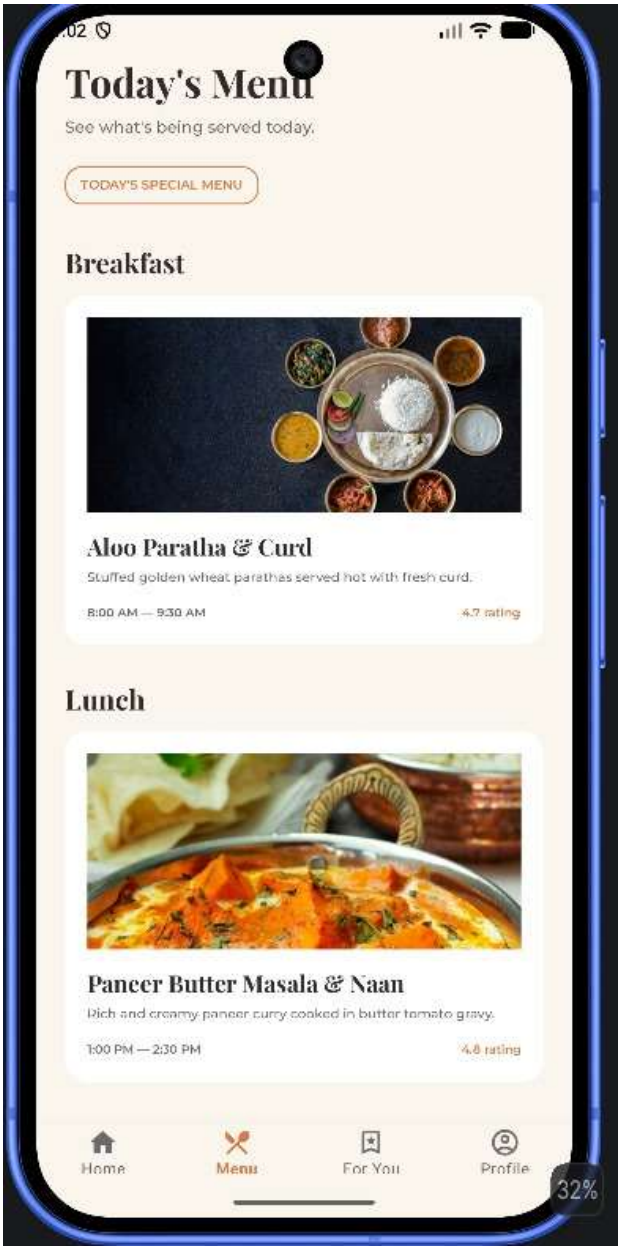
The home screen highlights today's special, previews meal times, and displays a recommended meal.



Source: screenshot supplied by the project team.

Figure 3. Today's Menu

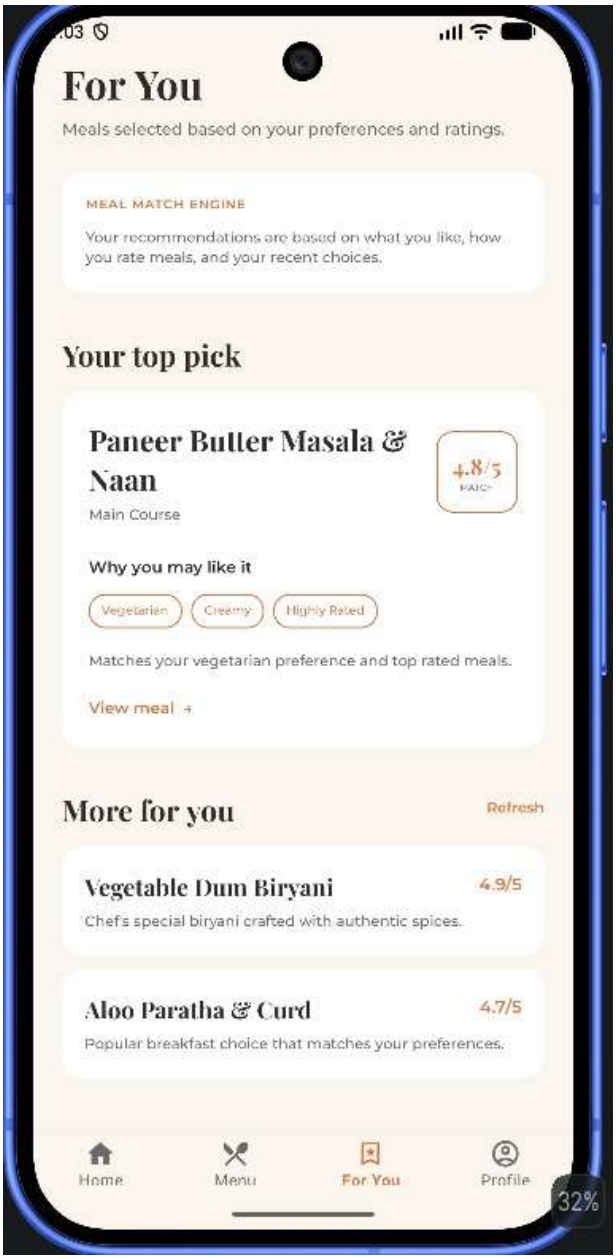
Meals are grouped by category, with images, descriptions, serving times, and ratings.



Source: screenshot supplied by the project team.

Figure 4. For You recommendations

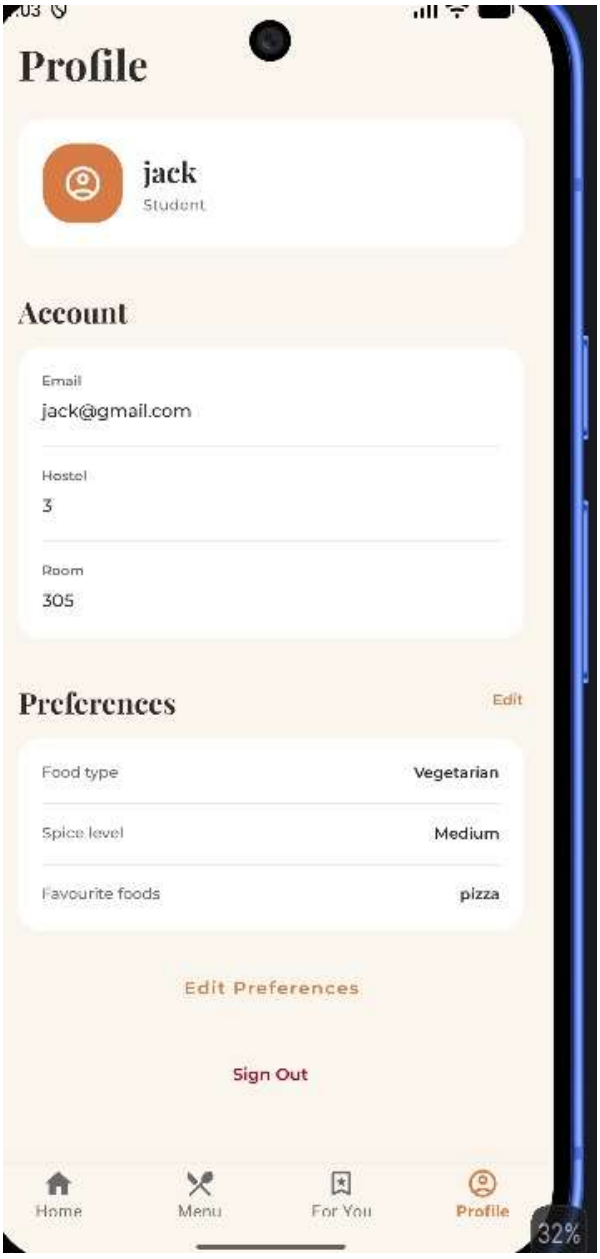
The screen presents a top meal pick, match score, preference-related reasons, and additional suggestions.



Source: screenshot supplied by the project team.

Figure 5. Profile and preferences

The profile screen displays account details, saved food preferences, preference editing, and sign-out controls.



Source: screenshot supplied by the project team.